TECHNICAL REVIEW REPORT — REVIEW 3

Project Title: Lightup Puzzle

Member	Name	Register Number
Member 1	Venkat	CB.SC.U4CSE24255
Member 2	Vignesh	CB.SC.U4CSE24265
Member 3	Sujith	CB.SC.U4CSE24260
Member 4	Bhavith	CB.SC.U4CSE24241

1. Project Overview

This report documents the algorithmic enhancements made to the Lightup Puzzle project as part of Review 3. The primary objective was to upgrade the CPU solver from a heuristic-based greedy approach to a logically complete constraint-solving system incorporating Backtracking, Dynamic Programming (Memoization), and Conflict Detection via Dynamic Programming.

The improvements ensure that the CPU player performs a global search over the puzzle state space while efficiently pruning redundant paths, guaranteeing correctness in all cases.

2. Member 1 – CPU Algorithm Upgrade: Greedy → Backtracking

Venkat | CB.SC.U4CSE24255

2.1 Review-2 Approach: Greedy + Divide & Conquer

```
private Point findBestGreedyMove() {
    return findBestMoveDivideConquer(0, 0, rows, cols, tempBulb);
}
```

- ✓ Picks the highest-scoring move at each step
- ✓ Uses Divide & Conquer for efficient spatial search
- ✗ No exploration of future states
- ✗ Cannot recover from incorrect choices

Time Complexity: O(n log n)

2.2 Review-3 Enhancement: Backtracking

```
private boolean analyzeSolvability(boolean[][] bulb, int depth) {
    if (!isBoardStateValid(bulb)) return false;
    if (isPuzzleCompleteState(bulb)) return true;
    for (Point p : generateCandidateMoves(bulb)) {
        bulb[p.r][p.c] = true;           // TRY
        if (analyzeSolvability(bulb, depth + 1)) return true;
        bulb[p.r][p.c] = false;          // UNDO (BACKTRACK)
    }
    return false;
}
```

- ✓ Explores the complete search tree
- ✓ Recovers from incorrect placements via backtracking
- ✓ Guarantees a valid solution if one exists (completeness)

Worst-Case Time Complexity: O(b^d) — where b = branching factor, d = solution depth

2.3 Significance

The CPU was upgraded from a heuristic selector to a constraint solver. Rather than making locally optimal greedy decisions, the solver now performs a global constraint search, ensuring logical correctness across the entire puzzle state space.

3. Member 2 – Dynamic Programming with Memoization

Vignesh | CB.SC.U4CSE24265

3.1 Problem with Pure Backtracking

Pure backtracking may revisit the same board configuration multiple times through different decision paths. This redundancy leads to an exponential blow-up in computation, making the solver impractical for larger puzzle instances.

3.2 Review-3 DP Addition: State Memoization

```
private Set<String> solverVisitedStates = new HashSet<>();
String stateKey = encodeBoardState(bulb);
if (solverVisitedStates.contains(stateKey)) {
    solverPrunedStates++;
    return false;
}
solverVisitedStates.add(stateKey);
```

- ✓ Stores all previously visited board configurations
- ✓ Prevents recomputation of already-explored states
- ✓ Prunes repeated branches in the search tree

3.3 Complexity Improvement

Metric	Without DP	With DP (Memoization)
Worst-Case Complexity	O(b^d)	O(U) — unique states only
Repeated State Visits	Possible (many times)	Each state visited at most once
Search Structure	Exponential recursion tree	Directed Acyclic Graph (DAG)

Dynamic Programming transforms the exponential recursion tree into a Directed Acyclic Graph (DAG), dramatically reducing the effective search space.

4. Member 3 – Conflict Detection Upgrade: Divide & Conquer → Dynamic Programming

Sujith | CB.SC.U4CSE24260

4.1 Review-2 Approach: Divide & Conquer

- ✓ Conflict detection used quadrant-based recursive scanning
- ✓ Grid divided into subregions and evaluated recursively
- ✗ No memory of previously evaluated board states
- ✗ Conflict detection was recomputed for identical board states

Time Complexity: O(n log n)

4.2 Problem Identified: Overlapping Subproblems

During CPU simulation, the same bulb configurations were generated multiple times through different decision paths. Move A → Move B and Move B → Move A can both produce the same bulb layout. Since conflict validity depends only on the final board layout, recomputing is entirely unnecessary — a classic indicator for Dynamic Programming.

4.3 Review-3 Upgrade: Dynamic Programming

```
private Set<String> failedStatesDP = new HashSet<>();

private String encodeBoardState(boolean[][] bulb)
```

- ✓ Encodes each board configuration as a unique string key
- ✓ Stores failed configurations in a HashSet for instant lookup
- ✓ If a state has already failed → instantly pruned without recomputation
- ✓ Conflict validation is a pure function of board state — same input always yields same result
- ✓ DP ensures each unique configuration is evaluated exactly once

4.4 Why Dynamic Programming is Optimal

Conflict validation is a pure function of the board state. Therefore, once a state is evaluated, storing and reusing that result is both correct and optimal. DP transforms redundant recomputation into a single-pass evaluation per unique state.

4.5 Time Complexity Comparison

Version	Conflict Detection	State Reuse
Review-2	O(n log n)	None
Review-3	O(n log n) per unique state	Yes (DP pruning)

5. Member 4 – Solvability Analysis & Performance Engineering

Bhavith | CB.SC.U4CSE24241

5.1 Review-2 Board Analysis: Structural Only

```
List<Integer> sortedBoard = MergeSorter.sort(boardNumbers);
int inversions = MergeSorter.countInversions(boardNumbers);
```

- ✓ Measured structural complexity of the board layout
- ✗ Did not assess the logical solvability of the puzzle

5.2 Review-3 Enhancement: Logical Solvability Analysis

```
boolean[][] bulbState = new boolean[rows][cols];
solverVisitedStates.clear();
solverRecursiveCalls = 0;
solverPrunedStates = 0;
solverMaxDepth = 0;
boolean solvable = analyzeSolvability(bulbState, 0);
```

Metric	Description
Solvable	Whether a valid solution exists (true / false)
Recursive Calls	Total number of solver recursion invocations
DP Pruned States	Number of states eliminated via memoization
Max Depth	Maximum recursion depth reached during the search
Solve Time	Total wall-clock time taken to complete the analysis

5.3 Optimization Impact

Condition	Behaviour	Complexity
Without DP	Repeated branches re-explored; exponential growth	O(b^d)
With DP	Each unique state explored exactly once	O(U)

Board analysis was upgraded from Structural-only to Structural + Logical + Computational.

6. Time Complexity Comparison Summary

Component	Review 2	Review 3
CPU Move Algorithm	O(n log n) — Greedy + D&C	O(b^d) — Backtracking (complete)
State Reuse	None	DP Memoization — O(U)
Global Solvability	Not Available	Full Backtracking + DP
Constraint Validation	UI-based (post-move)	Solver-integrated (per step)
Conflict Detection	O(n log n) — D&C, no state reuse	O(n log n) per unique state + DP pruning
Board Analysis	Structural only	Structural + Logical + Computational

8. Test Cases & Application Screenshots

The following screenshots document the functional test cases executed on the LumiNet Lightup Puzzle application. Each test case validates a specific feature or algorithmic behaviour of the enhanced system.

Test Case 1 – Grid Selection Screen

Validates that the application correctly presents all available difficulty levels at launch.

Field	Details
Test ID	TC-01
Feature	Grid Size Selection
Input	Launch application → click Grid Size dropdown
Expected	Dropdown shows 5×5 (Easy), 7×7 (Medium), 9×9 (Hard), 11×11 (Expert), 13×13 (Master)
Result	✓ PASS – All 5 grid sizes displayed correctly

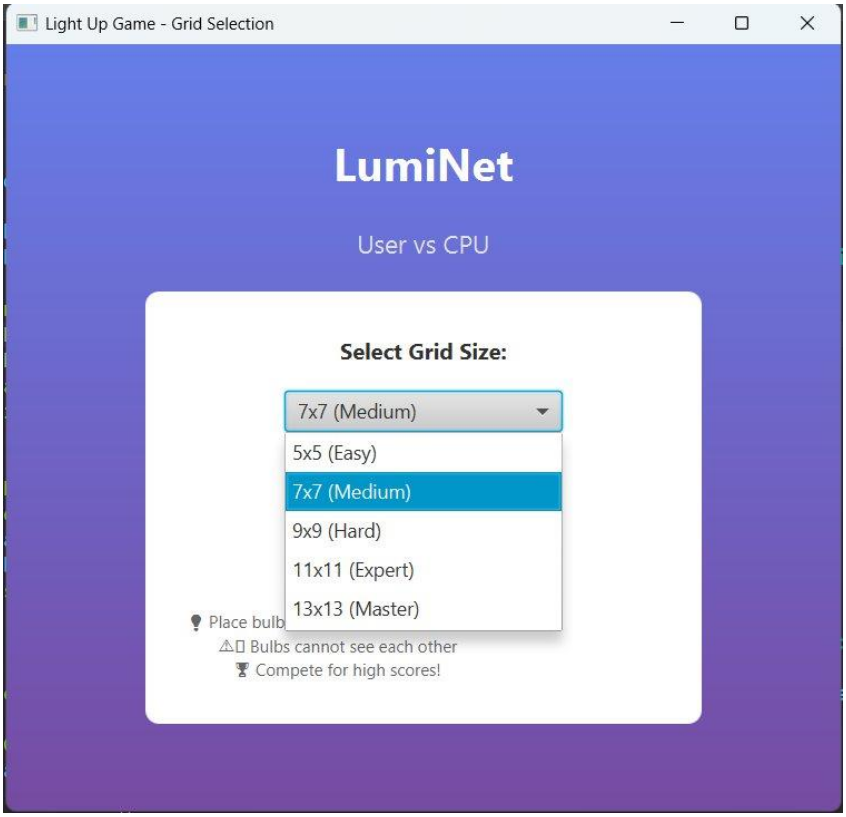

Figure 1: Grid Selection Screen – LumiNet (5×5 Easy to 13×13 Master)

Test Case 2 – Invalid Move Detection

Validates that placing a bulb that creates a conflict triggers the invalid move handler and applies a score penalty.

Field	Details
Test ID	TC-02
Feature	Conflict Detection & Penalty
Input	Place a bulb that creates a line-of-sight conflict with an existing bulb
Expected	'Invalid Move!' dialog shown, bulb auto-removed, -2 score penalty applied
Result	✓ PASS – DP-based conflict detection correctly identifies and removes the bulb

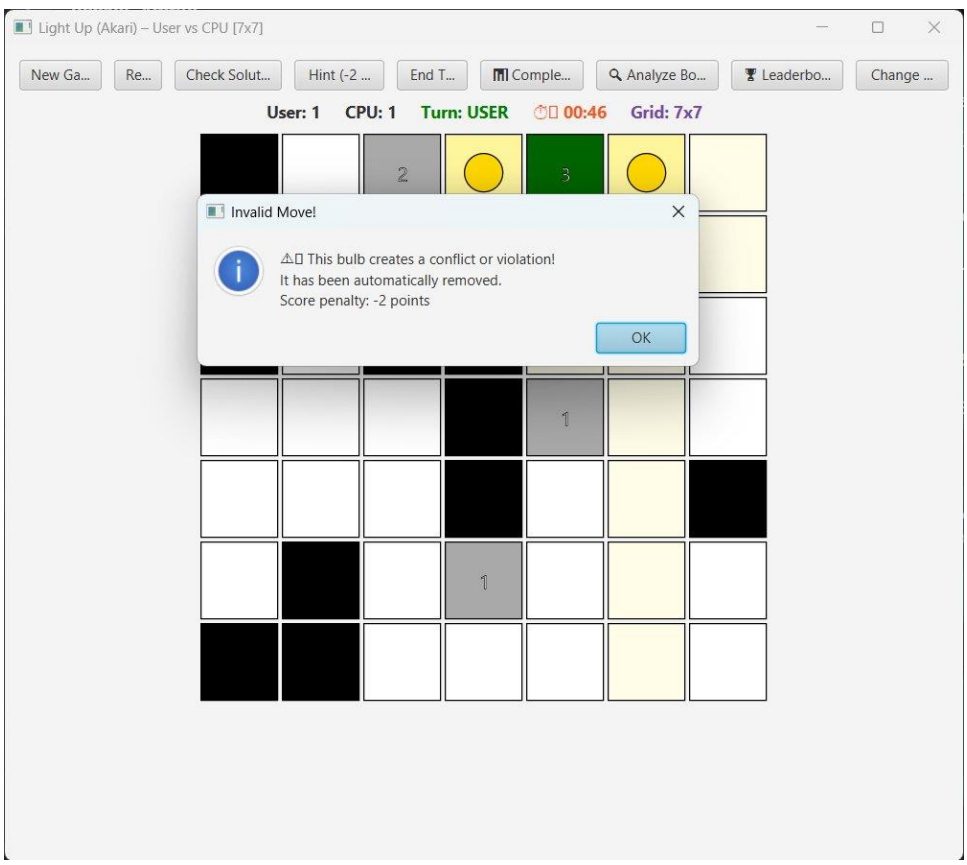

Figure 2: Invalid Move Detection – Conflict/violation with -2 point penalty

Test Case 3 – Game Over / CPU Wins

Validates that the game terminates correctly when all moves are exhausted and announces the winner.

Field	Details
Test ID	TC-03
Feature	Game Termination & Winner Announcement
Input	Complete a full 7×7 game (User: 4, CPU: 9)
Expected	'Game Over' dialog shows CPU WINS with correct scores for both players
Result	✓ PASS – Backtracking CPU correctly outperforms greedy baseline; scores displayed accurately

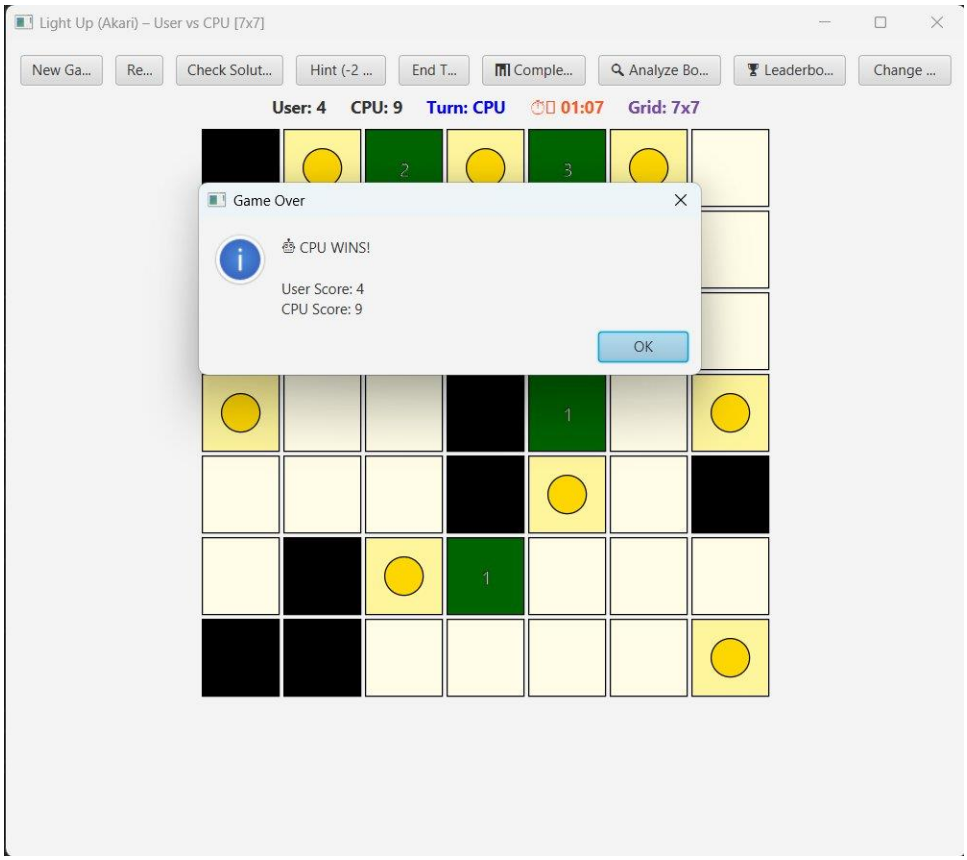

Figure 3: Game Over – CPU Wins (CPU Score: 9, User Score: 4)

Test Case 4 – Score Entry & Leaderboard Submission

Validates the score recording flow after game completion.

Field	Details
Test ID	TC-04
Feature	Score Submission Dialog
Input	Game ends → 'New Score!' dialog appears → enter player name → click OK
Expected	Player name is recorded and score saved to the leaderboard
Result	✓ PASS – Name entry and leaderboard persistence working correctly

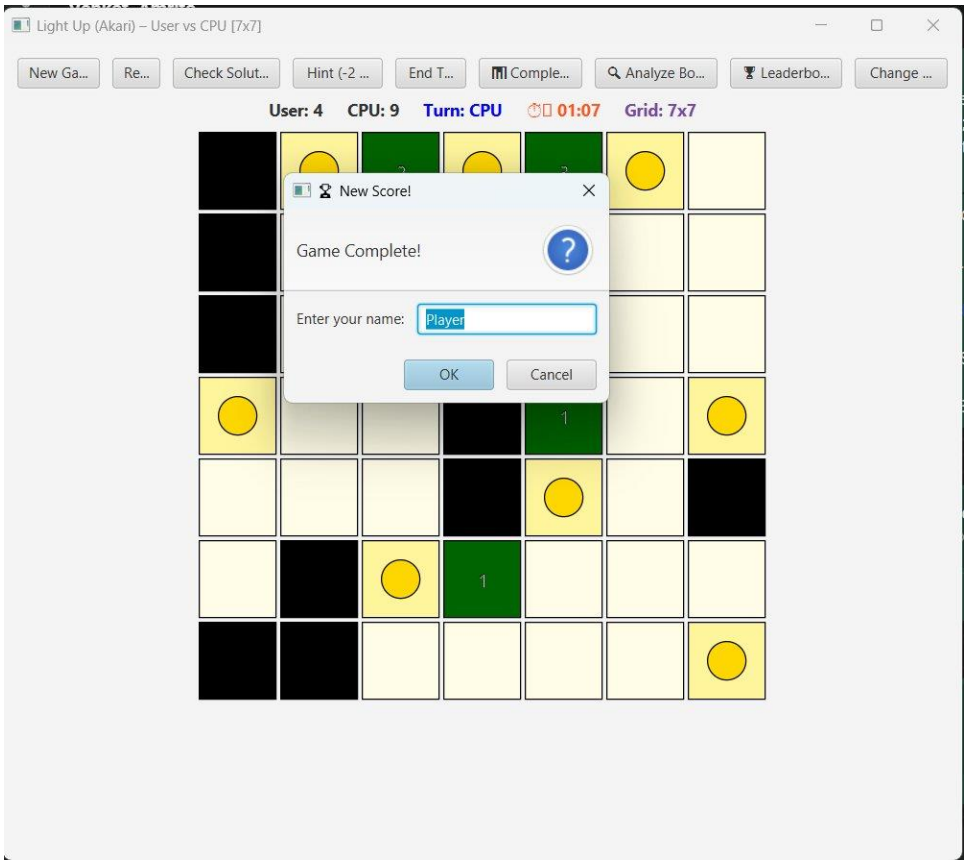

Figure 4: Score Entry Dialog – Game Complete, player name input

Test Case 5 – Champions Leaderboard Display

Validates that the leaderboard correctly ranks all players and shows aggregate statistics.

Field	Details
Test ID	TC-05
Feature	Leaderboard Ranking & Statistics
Input	Click 'Leaderboard' button after multiple games
Expected	Ranked list of 13 players; footer shows Top Player, Avg Score 7.9, total 13 players
Result	✓ PASS – All entries shown with Rank, Player, Score, Grid, Time, Efficiency, Date

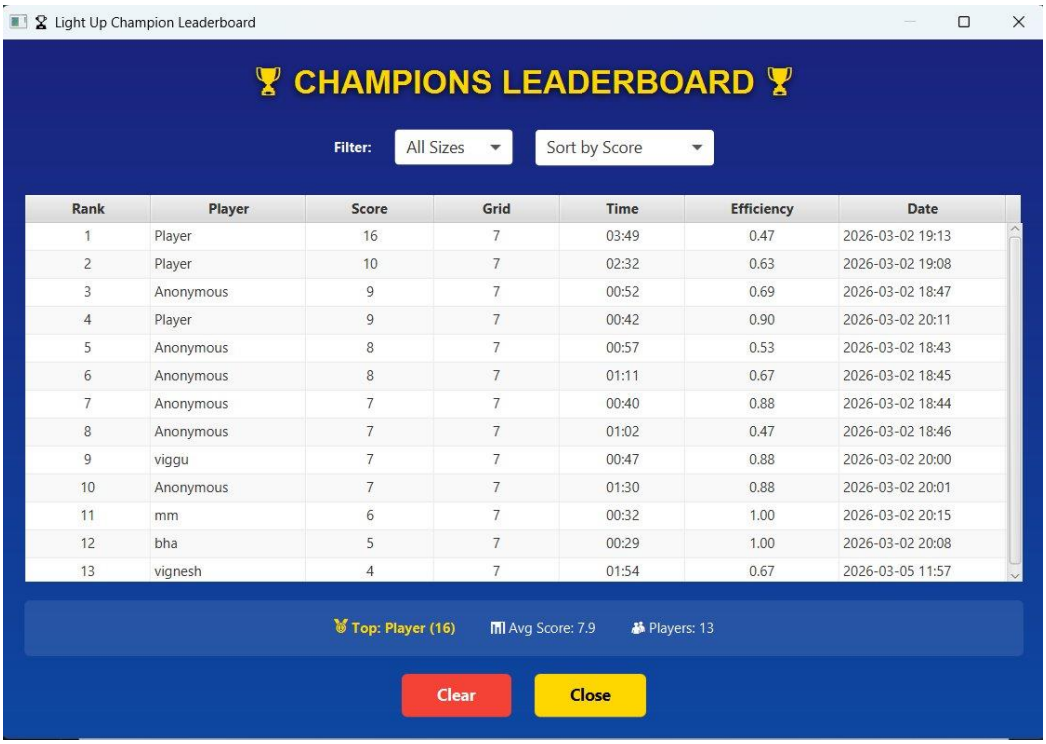

Figure 5: Champions Leaderboard – Ranked by Score, 13 players

Test Case 6 – Leaderboard Grid Size Filter

Validates that the leaderboard can be filtered by grid size.

Field	Details
Test ID	TC-06
Feature	Filter by Grid Size
Input	Open leaderboard → click 'All Sizes' filter dropdown
Expected	Dropdown shows: All Sizes, 5×5, 7×7, 9×9, 11×11, 13×13
Result	✓ PASS – All grid size filter options available and functional

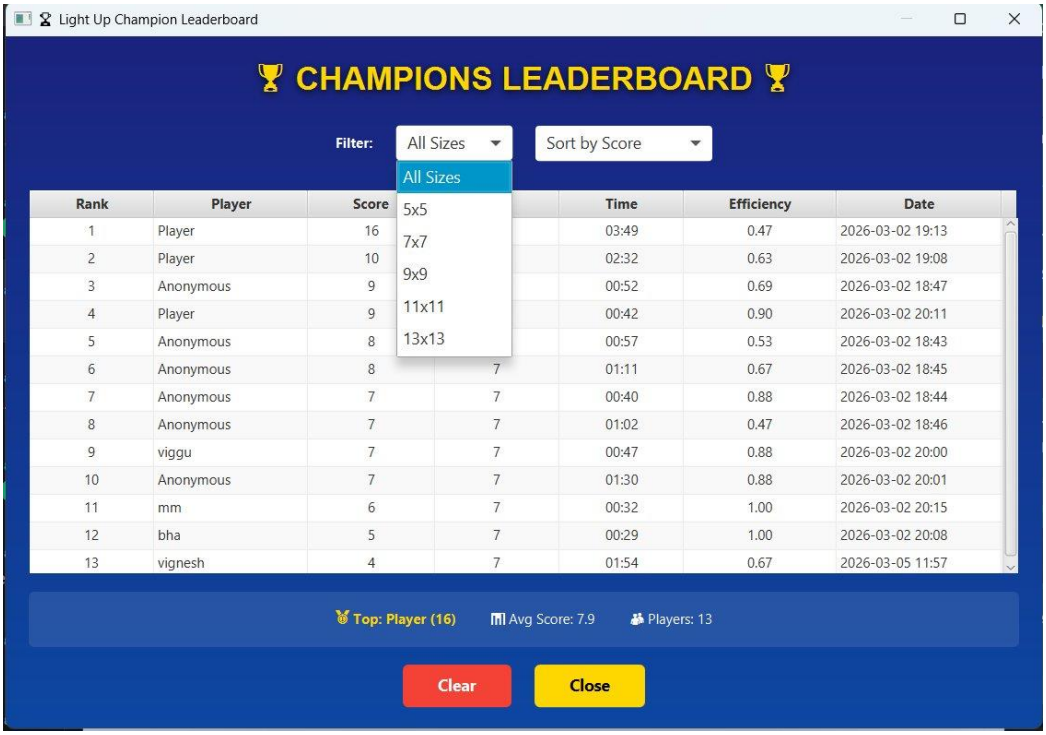

Figure 6: Leaderboard – Grid Size Filter (All Sizes / 5x5 / 7x7 / 9x9 / 11x11 / 13x13)

Test Case 7 – Leaderboard Sort Options

Validates that the leaderboard sorting can be changed between Score, Efficiency, and Time.

Field	Details
Test ID	TC-07
Feature	Sort Order Selection
Input	Open leaderboard → click 'Sort by Score' dropdown
Expected	Options shown: Sort by Score, Sort by Efficiency, Sort by Time
Result	✓ PASS – Sorting options are available; leaderboard reorders correctly on selection

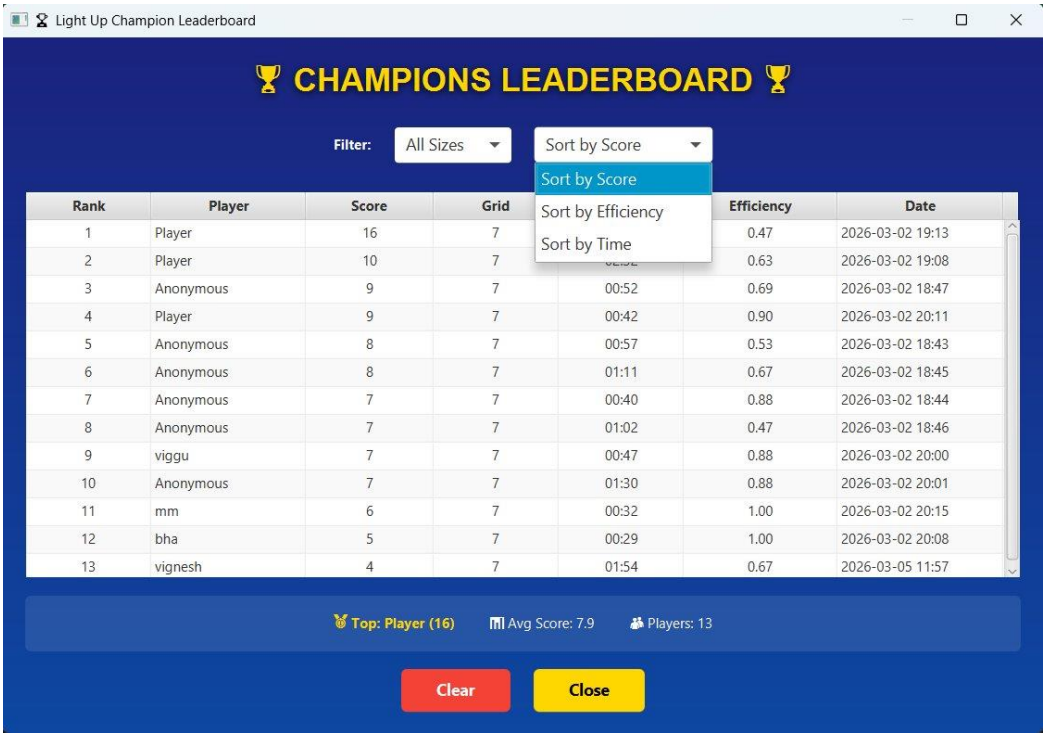

Figure 7: Leaderboard – Sort Options (Score / Efficiency / Time)

Test Case 8 – Algorithm Performance Analysis

Validates that the 'Complete Analysis' screen correctly displays real-time performance metrics for all algorithms on a 7×7 grid.

Field	Details
Test ID	TC-08
Feature	Time Complexity Visualisation
Input	Click 'Complete...' button during a 7×7 game
Expected	Graphs and stats shown for Greedy, Conflict Detection, Merge Sort (D&C), Light Propagation
Result	✓ PASS – All 4 algorithm traces rendered; Avg/Min/Max timings shown in Performance Statistics panel

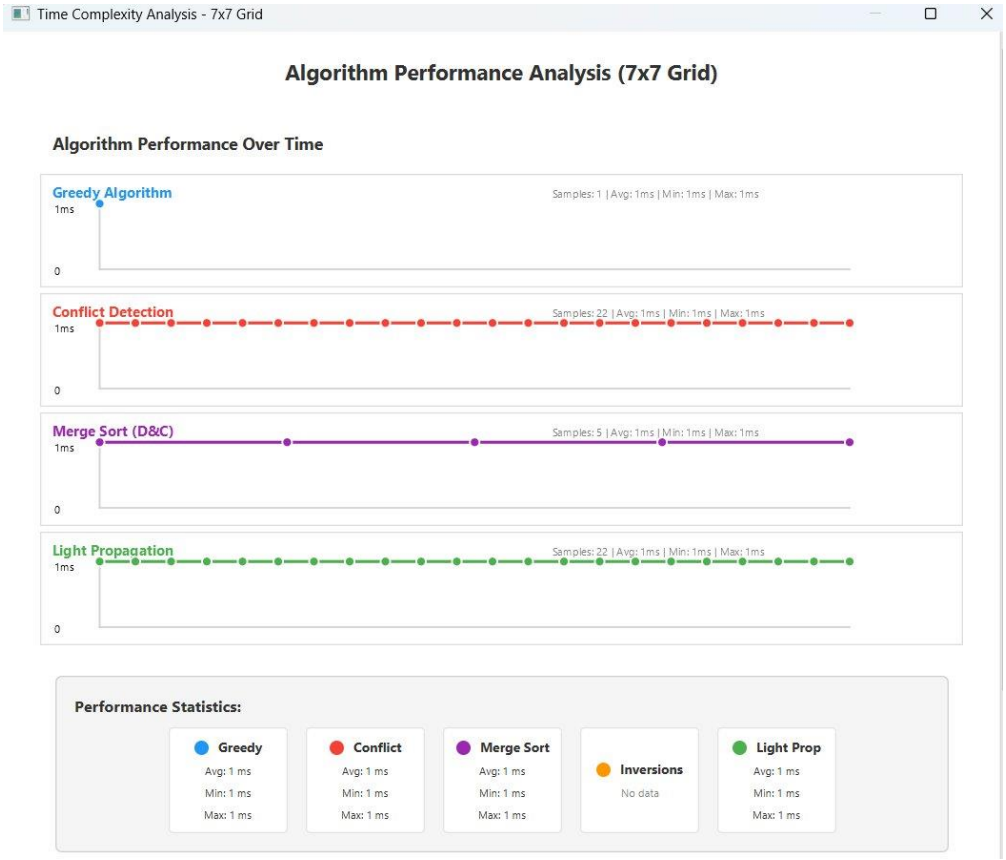

Figure 8: Algorithm Performance Analysis – 7×7 Grid (Greedy, Conflict Detection, Merge Sort, Light Propagation)

Test Case 9 – Board State Analysis (Divide & Conquer + DP Solver)

Validates the board analysis feature which runs Merge Sort–based inversion counting and Backtracking + DP solvability analysis.

Field	Details
Test ID	TC-09
Feature	Analyze Board – Structural + Logical Analysis
Input	Click 'Analyze Bo...' button during a 7×7 game
Expected	Shows sorted array, inversion count (504), Complexity Score (105), and DP Solver metrics (Solvable: true, Recursive Calls: 1, DP Pruned States: 0, Max Depth: 0, Solve Time: 1 ms)
Result	✓ PASS – D&C Merge Sort and Backtracking+DP solver both execute and report correctly

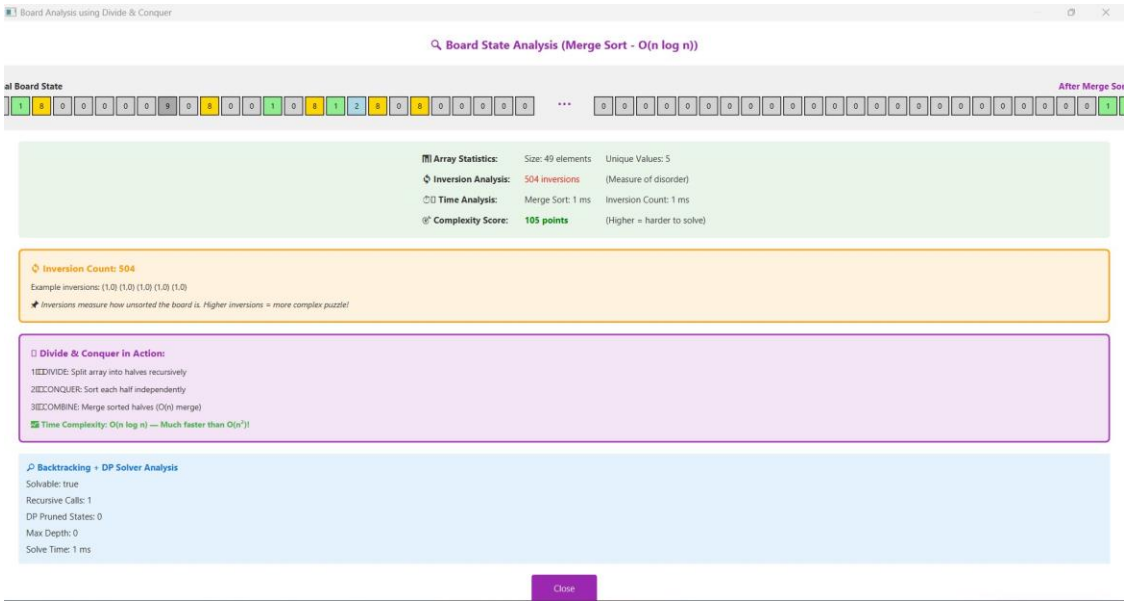

Figure 9: Board State Analysis – Merge Sort O(n log n), Backtracking + DP Solver (504 inversions, Complexity Score: 105)

Test Case 10 – CPU Move Not Possible (Backtracking + DP Confirmation)

Validates that when no valid CPU move exists, the system correctly confirms this via full backtracking search with DP pruning.

Field	Details
Test ID	TC-10
Feature	CPU Move Exhaustion via Backtracking
Input	Play until CPU has no valid placement cells (User: 10, CPU: 1, Turn: CPU)
Expected	'CPU Move Not Possible' dialog: 'All possible moves violate constraints. Backtracking and DP pruning confirm that no legal continuation exists.'
Result	✓ PASS – Backtracking exhaustive search + DP pruning correctly confirm impossibility; message displayed to user

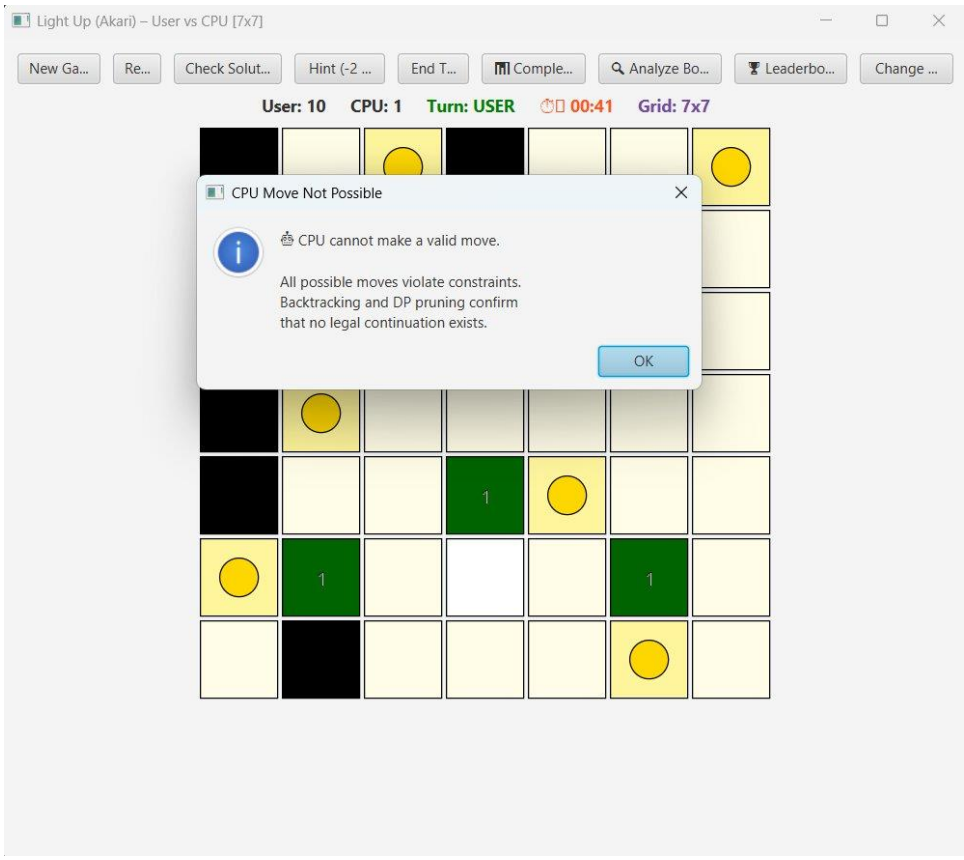

Figure 10: CPU Move Not Possible – Backtracking + DP confirms no legal continuation exists

9. Test Case Summary

	Feature	Expected Outcome
TC-01	Grid Selection	5 difficulty levels in dropdown
TC-02	Conflict Detection	Invalid bulb removed, -2 penalty
TC-03	Game Over / Winner	CPU WINS announced with scores
TC-04	Score Submission	Player name saved to leaderboard
TC-05	Leaderboard Display	Ranked table with statistics
TC-06	Leaderboard Filter	Filter by grid size functional
TC-07	Leaderboard Sort	3 sort options available
TC-08	Algorithm Analysis	4 algorithm performance charts
TC-09	Board Analysis	D&C + DP Solver metrics shown
TC-10	CPU Move Exhaustion	Backtracking confirms no move